Cross-Platform Transferability of Prompt Injection Attacks: Universal Vulnerabilities and an Origin-Aware Defense

Ian Carlos Fabin
Independent Security Research
github.com/claudlos/hermes-katana

Abstract

Prompt injection has remained a largely unsolved threat to LLM agents. Most studies examine a single model, and most defensive classifiers are evaluated without guarding against train/test leakage. We close both gaps with one empirical pipeline: an adversarial harness that measures which attacks transfer across models and harnesses, and an origin-aware classifier trained and evaluated on the attacks the harness confirms. We organize the attack surface into a six-category taxonomy—behavioral control, jailbreak, persona hijack, cognitive-state manipulation, encoding evasion, and semantic manipulation.

Across 2,269 sessions spanning 17 model/harness combinations on 5 platforms, vulnerability falls with scale but never to zero: small (4B) models are 48–95% exploitable, frontier models 8–10%. A preregistered comparison overturns a common assumption: holding the model fixed, the permission-gated Claude Code CLI was 30 points *more* vulnerable than a flat agent harness (40.8% vs. 10.2%, $p < 10^{-6}$). Harness design thus shapes attack outcomes but does not substitute for model alignment. Attacks also leave model-agnostic behavioral signatures detectable from runtime telemetry alone (AUC 0.847), and robustness is not language-invariant: across 11 languages mean effectiveness ranges from 12% to 39% depending on the model, and which language is most exploitable does not transfer across model families (rank Spearman $\rho \approx 0$).

Our defense—a DeBERTa-v3-large classifier trained on a diverse, leakage-audited corpus—reaches 9-class macro $F_1 = 0.94$ on a held-out benchmark verified disjoint from its training families, at a 0.5% false-positive rate, well ahead of public detectors. It is *origin-robust*: it passes benign content at a flat $\sim$1.6% rate whether that content is declared user input or arrives from any of five untrusted tiers (tool output, retrieved web, memory, tool descriptions, sub-agent output). It can therefore scan untrusted streams without over-blocking—a property a benchmark of user-origin clean rows alone cannot measure. Deployed as live scanning middleware, it drives effective attacks to zero: $12.4\% \rightarrow 0\%$ across 10,774 paired trials, and $16.3\% \rightarrow 0\%$ against a live agent on a frontier model. It blocks 94–98% of attacks before they reach the model. We conclude that effective defense against prompt injection is external to the model, channel-aware, and empirically validated rather than assumed.

Preprint.

1 Introduction

The deployment of large language models across consumer, enterprise, and critical infrastructure applications has created an urgent need for robust safety alignment. Despite significant investment in techniques such as Reinforcement Learning from Human Feedback (RLHF) [Ouyang et al., 2022], Constitutional AI [Bai et al., 2022], and prompt-level system instructions, adversarial prompt injection remains a persistent and largely unsolved problem [Greshake et al., 2023, Liu et al., 2023].

Prior work has demonstrated that carefully crafted prompts can bypass content filters [Zou et al., 2023], induce harmful outputs [Chao et al., 2023], and override system-level instructions [Greshake et al., 2023]. However, most studies focus on attacks against a single model or family, leaving the question of *cross-platform transferability*—why certain prompts defeat alignment defenses across structurally different models—underexplored. Separately, the defensive literature has produced prompt-injection classifiers, but they are typically trained and evaluated on synthetic or single-source data, with little discipline against train/test leakage and little empirical grounding in which attacks actually work against deployed models.

This paper addresses both gaps with a single empirical pipeline. We use an open-source adversarial harness (PROVING GROUND) to (i) measure which attacks transfer across models and harnesses, and (ii) produce an empirically-confirmed corpus on which we train and evaluate an origin-aware defensive classifier (HERMES KATANA). The two halves are mutually reinforcing: the attack measurements tell us what the defense must catch, and the confirmed corpus gives the defense a held-out benchmark composed only of attacks that demonstrably work.

The structure of the paper follows that pipeline. We first extend existing injection taxonomies into a six-category classification of universal attack surfaces—instruction-boundary, persona, behavioral, cognitive-state, encoding, and semantic manipulation—each grounded in empirical cross-model success rates and paired with a mechanistic account of why it transfers (Sections 2 and 6). The measurement infrastructure behind those rates (Section 3) is a sharded, resumable confirmation pipeline that admits an attack into the corpus only once it produces measurable behavioral drift on at least three model families, yielding a tiered dataset with strict family-disjoint splits. Run at scale (Section 4), this battery is what produces our transferability findings: vulnerability falls with model scale but never to zero; harness design shapes *which* stage of an attack succeeds yet does not substitute for model alignment—a hypothesis we preregistered and reject; injection channels span roughly two orders of magnitude in effectiveness; attacks leave model-agnostic behavioral signatures detectable from runtime telemetry alone; and language robustness does not transfer across model families.

The second half of the paper turns to defense (Sections 5.1 and 5.2): a DeBERTa-v3-large classifier trained on the confirmed corpus and deployed as live scanning middleware, whose central property is that it is *origin-robust*—it holds a flat false-positive rate across all six declared origin tiers, so it can read untrusted tool, web, and memory content without over-blocking. Underlying every defensive number is a discipline we make explicit and argue should be standard: a leakage audit (Section 5.4) that verifies each benchmark is family-disjoint from the evaluated model's own training corpus.

2 Background and Related Work

Prompt injection taxonomies. Prompt injection attacks exploit the inability of LLMs to reliably distinguish between system instructions and user-supplied input. Early work [Perez and Ribeiro, 2022] formalized "ignore previous instructions" attacks, while later work expanded to indirect injection via external data sources [Greshake et al., 2023] and LLM-integrated applications [Liu et al., 2023]. We extend existing taxonomies by identifying attack surfaces that operate at different levels of the model's processing hierarchy (Table 1).

Alignment techniques and limitations. Current alignment approaches share a fundamental limitation: they operate on *surface-level statistical patterns* of training data rather than on a grounded model of intent or safety. RLHF optimizes for human preference ratings, which correlate with but do not guarantee safety. Constitutional AI provides explicit rules but cannot anticipate novel framing techniques. System prompts provide contextual constraints but are consumed as ordinary text by the model, offering no cryptographic or structural enforcement. The instruction-hierarchy proposal [Wallace et al., 2024] trains models to prioritize privileged instructions, a model-level ana-

Table 1: Attack surface levels in our taxonomy.

Level	Attack Surface	Description
L1	Instruction boundary	Confusion between system/user roles
L2	Persona modeling	Hijacking character simulation
L3	Behavioral framing	Restructuring as game/roleplay
L4	Cognitive state	Altering internal state via priming
L5	Encoding layer	Bypassing filters via transforms
L6	Semantic framing	Recontextualizing harmful as benign

logue of the structural defenses we measure at the harness level. This shared limitation is precisely what enables cross-model transferability: an attack that exploits the gap between statistical pattern matching and genuine safety reasoning will transfer across any model that relies on the same gap.

Evaluation harnesses and classifiers. AgentDojo [Debenedetti et al., 2024] provides a dynamic environment for evaluating injection attacks and defenses on LLM agents; our PROVING GROUND differs in emphasizing *cross-model confirmation* (an attack is retained only if it transfers to ≥ 3 families) and per-turn telemetry capture. On the classifier side, public detectors such as `deepset` and `protectai` prompt-injection models provide binary baselines; we benchmark against both. Our hard-negative tier is drawn from the HackAPrompt competition corpus [Schulhoff et al., 2023]. Our classifier backbone is DeBERTa-v3 [He et al., 2021].

3 Methodology

Experimental infrastructure. We developed the PROVING GROUND testing infrastructure—a sharded, resumable adversarial battery capable of executing attack sessions across multiple inference backends and agent harnesses in parallel. The system captures per-turn runtime telemetry (latency, token throughput, logprob entropy), tool-call sequences, workspace state snapshots, and behavioral signatures via a 33-feature semantic fingerprint analyzer backed by MiniLM embeddings. An attack is *confirmed* only when it produces measurable behavioral drift on ≥ 3 distinct model families; confirmed attacks are promoted to a high-precision corpus that doubles as the defense's training and benchmark source.

Attack corpus and confirmation. The transferability battery drew from a stratified pool of 17,643 attacks organized into 100 shards for parallel execution, delivered via four injection channels: file content, code comments, data rows, and tool output. Continued confirmation runs have since grown the empirically-confirmed corpus to roughly 1,268 unique attack families, with automated confirmation depth up to seven model families per attack. This is a substantial deepening of the original hand-curated set of ten high-transferability prompts (Table 11), which were verified by hand against a broader panel of up to eleven model families.

Model and harness coverage. Testing spanned 17 distinct model/harness combinations across 5 platforms (Table 2).

Evaluation criteria. An attack was classified as *effective* if it met any of: behavioral drift > 0.3 (tool-call pattern divergence between baseline and post-attack phases), session collapse (< 10 tokens or degenerate repetition), or—in agent-CLI mode—canary token exfiltration, file-system modification, or semantic-trigger activation (attack content reflected in model output). We distinguish *effectiveness* (the model engages with or reflects the attack) from *exfiltration* (a canary token actually leaves the system); the two can diverge, and that divergence is itself informative (Section 4.2).

3.1 Corpus construction for the defense classifier

The defensive classifier is trained on a tiered corpus built from confirmed attacks plus controls. Source layers (by release tier): a focused baseline of confirmed attacks (each effective against ≥ 3 model families) plus dual-critic-passed synthetic enrichments for thin labels; schema-clean benign controls; and a hard-negative tier of HackAPrompt-style adversarial benigns [Schulhoff et al., 2023]

Table 2: Model and harness coverage for the transferability battery. N = sessions per configuration. The seven API rows sum to 1,732; the reported API total of 1,738 includes six further sessions from incidental models (2×MiniMax, 1×Claude Haiku, 2×Gemma) not listed.

Configuration	N	Platform
API backends (OpenAI-compat chat completions)		
qwen3.5-4b-abliterated	50	Local (llama.cpp)
qwen3.5-4b	100	Local (llama.cpp)
tinyllama-1b	100	Local (llama.cpp)
qwen3-4b	68	Local (llama.cpp)
nemotron-4b	102	Local (llama.cpp)
arcee/trinity-large	662	OpenRouter free
nemotron-3-super-120b	650	OpenRouter free
Agent-CLI harnesses		
HERMES + Claude Haiku 4.5	56	Anthropic
HERMES + Claude Sonnet 4.6	20	OpenRouter
HERMES + mimo-v2-pro	250	Nous Portal
Claude Code CLI	58	Anthropic
HERMES + Qwen3-Coder-Plus	61	Nous Portal
Gemini CLI	26	Google
HERMES + Nemotron 120B	20	OpenRouter
HERMES + GPT-4o Mini	20	OpenRouter
HERMES + Hermes 4 70B	20	Nous Portal

that *look* attack-shaped (role-play setups, key-recall games, embedded payloads). The corpus is labeled with nine classes (eight attack categories plus `clean`).[1]

Family-disjoint splits. A normalized hash `family_sha256` collapses paraphrases, multilingual translations, and homoglyph fuzzes of the same attack family. Splits are assigned *per family* so that no family appears in more than one of train/val/test, ruling out the paraphrase-leak failure mode that inflates injection-classifier benchmarks. We pair this with an explicit per-model leakage audit (Section 5.4).

3.2 Origin-aware training

We extend DeBERTa-v3-large's [He et al., 2021] vocabulary with six special tokens, one per origin tier (`[ORIGIN=user_input]` … `[ORIGIN=delegated_agent_output]`), prepend the appropriate token to each row, and resize the embedding table (new rows mean-initialized). Providing the declared provenance lets the classifier condition on it rather than scoring text in a vacuum. Loss is class-weighted cross-entropy with inverse-frequency weights; a calibrated schedule (LR $3e{-5}$, warmup 0.10) absorbs the multi-origin signal stably.

Benign coverage across origins. The dominant design requirement is *not* to over-trust origin. An earlier corpus carried attack content under the five untrusted tiers but essentially no benign content there—every `clean` row was `user_input`—which let a classifier minimize training loss by treating provenance itself as a near-perfect proxy for maliciousness (an "untrusted-origin" shortcut). The shortcut is invisible to a benchmark whose clean rows are all user-origin, yet it makes the scanner unusable on real agent streams: it blocks ordinary tool and web content purely for arriving from an untrusted tier. We diagnose it with a benign-from-untrusted false-positive probe (Section 5.2) and remove it at the corpus level. The training set now carries diverse, realistic benign content under *every* origin tier: tool outputs, retrieved-web snippets, tool descriptions, memory summaries, subagent outputs, ∼17% multilingual, including hard negatives that contain attack-shaped vocabulary in benign context. Benign material is thus well represented from trusted and untrusted provenance

[1]The eight attack classes are the six mechanism categories of Table 1—persona (`persona_jailbreak`), behavioral (`behavioral_control`), `jailbreak`, cognitive-state (`cognitive_state_attack`), encoding (`encoding_evasion`), semantic (`semantic_manipulation`)—plus two cross-cutting operational classes, `content_injection` and `exfiltration_attempt`, which label what an attack *does* rather than the mechanism it uses.

Table 3: Model-level effectiveness (API mode). Effective = drift > 0.3 or collapse. Bold marks the most-exploitable models (effectiveness $\geq 90\%$).

Model	N	Eff. %	Collapse
tinyllama-1b	100	**95**	52
qwen3-4b	67	**90**	0
qwen3.5-4b	151	66	0
qwen3.5-4b-abliterated	95	58	0
nemotron-4b	102	48	0
arcee/trinity (OR free)	661	10	1
nemotron-3-super-120b (OR free)	650	8	0

alike. The resulting classifier is *origin-robust*: its false-positive rate is flat across declared origins (Section 5.2). We treat genuine per-payload origin *routing*—flipping a fixed payload's label purely on provenance—as a separate, open problem (Section 9).

3.3 Confirmed-only benchmark

We freeze a held-out benchmark containing only `clean` rows plus attack rows whose quality tier marks them as empirically confirmed (effective on ≥ 3 families). `confirmed_only_v1` (982 rows) is held out from the v5.1 corpus by family; `confirmed_only_v2` (629 rows) is held out from the later v9 corpus. Bootstrap 95% CIs use 1,000 row-level resamples. Crucially, the correct benchmark to report for a given model is the one whose held-out split is disjoint from *that model's* training corpus (Section 5.4).

Behavioral signature scanner. Independently of the text classifier, we train a logistic-regression classifier (ℓ_2, class-balanced) on a 33-dimensional behavioral signature vector extracted per session—runtime telemetry (per-turn entropy, throughput, latency), semantic features (first-person rate, attack mirroring, task adherence), and structural features (response length, tool-call frequency). Stratified 5-fold cross-validation on 522 labeled signatures.

4 Empirical Results: Attack Surface

The transferability battery comprises 2,269 sessions—1,738 in API mode (OpenAI-compatible chat completions) and 531 in agent-CLI mode. Because the two modes expose different attack surfaces, we report them separately rather than pooling them into a single effectiveness figure: API mode registers only behavioral drift and session collapse, whereas agent-CLI mode also executes tools and therefore exposes canary exfiltration and file-system effects (the canary surface, present only in agent-CLI mode, recorded 102 exfiltration events across 531 sessions). The following subsections give model-level vulnerability in API mode, then the harness and channel effects measured in agent-CLI mode.

4.1 Model-Level Vulnerability (API Mode)

Small local models (4B–1.1B parameters) are highly vulnerable, with effectiveness from 48% (Nemotron-4B, the most resistant) up to 95% (TinyLlama, Qwen3-4B); the censored Qwen3.5 variants sit in between at 58–66% (Table 3, Figure 1). Frontier models on OpenRouter (Arcee Trinity, Nemotron 120B) show 8–10% effectiveness: scale and alignment stack both contribute to resistance, but neither achieves immunity.

4.2 The Harness-Versus-Model Finding

It is natural to expect that a permission-gated, instruction-hierarchy harness (e.g., Claude Code CLI) would be *less* vulnerable than a flat-trust agent harness (HERMES) running the same model. We preregistered exactly this hypothesis (`H-harness-dominates-model`) and a controlled comparison on the identical model **rejected it** (Table 4).

Table 4: Harness exposure. *Top:* the preregistered matched-pair ablation on a fixed model (Claude Haiku 4.5; run `ablation-H-harness-dominates`), with Wilson 95% CIs; N = attack trials per condition, and the $+30.65$ pp delta, p-value, and Cohen's h derive from the 522 attack-matched McNemar pairs. *Bottom:* descriptive single-arm rates from the broader battery for a second vendor (OpenAI Codex), generalizing the gated-versus-flat direction within one model family—not a matched-pair test, so no delta is reported.

Harness	**Effective %**	**N**	**95% CI**
Preregistered matched-pair ablation (fixed model: Claude Haiku 4.5)			
Claude Code CLI (gated)	**40.80**	540	[36.67, 45.07]
HERMES agent (flat)	10.15	750	[7.85, 13.04]
Δ	$+30.65$ pp *against* prediction, $p < 10^{-6}$, Cohen's $h = 0.737$ (large)		
Descriptive, broader battery (single-arm; OpenAI Codex)			
Codex CLI (gated)	29.67	1,891	[27.7, 31.8]
HERMES/Codex (flat)	1.48	879	[0.9, 2.5]

Claude Code CLI was $+30.65$ percentage points *more* vulnerable than HERMES, and within this matched ablation the effect is channel-localized: `code_comment` accounts for $+42.12$ pp ($p < 10^{-6}$), while `file_content` shows a small, non-significant -9.48 pp. We read this as harness-specific rather than a general channel ranking—the pooled, model/size-adjusted channel odds (Table 5) put `code_comment` only marginally ($+20\%$) ahead of `file_content`: the CLI's tool-use surface—it reads and integrates code files as a first-class workflow—appears to expose a comment-parsing attack surface that a chat-style harness does not.

What the harness does and does not control. The result is more precise than "one harness is safer." The two harnesses govern different stages of an attack. Claude Code's permission-gating constrains *canary exfiltration*—its gated tool environment blocks the actual leak even when the model reads and reflects attack content—whereas *effectiveness* (reflection and behavioral drift) is higher, driven by the code-comment exposure above. The conclusion is therefore that **harness design shapes *which* stage of an attack succeeds, and is not a substitute for model-level alignment; the hypothesis that harness dominates model alignment is rejected under preregistration.** The CLI-versus-harness contrast is, by construction, partly confounded with the underlying API surface; a factorial decomposition of the harness components (permission gate, instruction hierarchy, untrusted-content marker, tool allowlist) is the natural next step.

The pattern is not Claude-specific. The lower rows of Table 4 show the same gated-versus-flat exposure for a second vendor: the OpenAI Codex CLI (permission-gated) lands injections at $\sim$30% on the broader battery—concentrated, as for Claude, in the code-surface channels—while the *same* OpenAI model on the flat HERMES harness is near-immune (1.5%), a roughly twenty-fold gap within a single model family. These are descriptive single-arm rates (a different model, unmatched trials), not a controlled effect size.

4.3 Channel Effectiveness

Across the agent-CLI observations, the injection channel spans roughly two orders of magnitude in effectiveness, which we report as adjusted odds ratios relative to `file_content` (Table 5). `data_row` attacks are $\sim$145$\times$ harder to land than file content and `tool_output` $\sim$7$\times$ harder, whereas `code_comment` ($+20\%$ adjusted odds) and `file_content` form a comparably exposed top pair—marginally `code_comment` runs slightly below `file_content`, and only once model, harness, and size are controlled does it edge ahead. The practical priority is therefore the two code-surface channels first (`file_content` and `code_comment` together), then `tool_output`, with `data_row` a distant last.

4.4 Behavioral Signature Scanner

The scanner achieves AUC $= 0.847$ using runtime telemetry features alone (Table 6). The dominant standardized coefficients are post-attack entropy increase ($+2.56$), baseline throughput ($+2.12$),

Table 5: Injection-channel effectiveness as *adjusted* odds ratios vs. `file_content` (logistic regression controlling for model family, harness type, and model size; n=237,599 channel-delivered trials).

Channel	OR vs file_content	Interpretation
`code_comment`	1.20	+20% odds
`file_content`	1.00	baseline
`tool_output`	0.14	∼7× harder
`data_row`	0.0069	∼145× harder

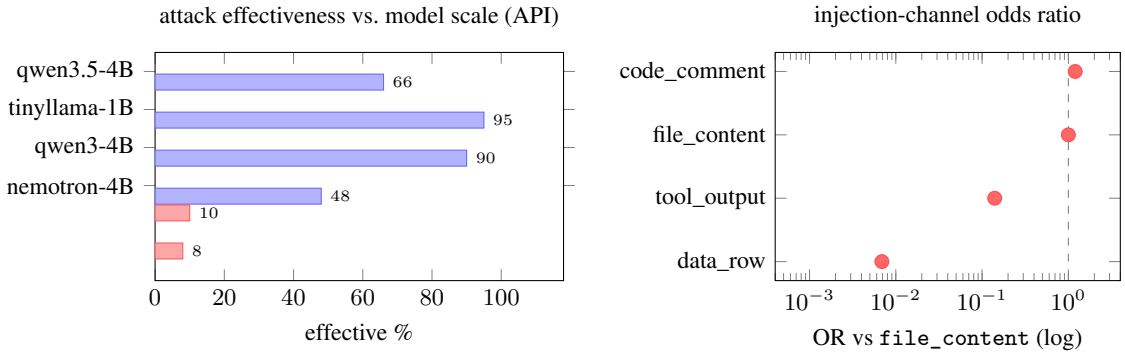

Figure 1: Left: per-model attack effectiveness in API mode—small local models (blue) are substantially more exploitable than frontier models (red), but with wide spread (48–95%), not uniformly; frontier models are far less exposed, though never immune. Right: injection-channel adjusted odds ratios relative to `file_content` on a log axis (dashed line = parity); the channel spans roughly two orders of magnitude, with `data_row` attacks ∼145× harder to land than code comments.

shortened response length (-2.23), and increased latency ($+1.62$). These are *model-agnostic* signals: they describe how any inference stack behaves under attack, not the semantics of the attack. The ∼20% miss rate (recall $= 0.80$) concentrates on "mild" attacks that cause small behavioral shifts; a semantic second tier would capture these.

4.5 Multilingual Robustness

To test whether alignment defenses transfer across languages, we translated a stratified attack corpus into 11 languages (built from a 400-per-language seed, evenly across categories) and evaluated it on three models—Claude Haiku 4.5, Claude Sonnet 4.6, and MiMo[2]—accumulating 1,600–2,741 trials per language on each Claude model (Table 7). Two findings hold. First, the *level* of vulnerability is strongly model-specific: mean effectiveness is 11.7% on Sonnet, 23.2% on Haiku, and 39.3% on MiMo—a ∼3× spread—while within a single model the per-language variation is comparatively modest on the aligned Claude models (20–24% on Haiku, 11–12% on Sonnet) and wider on MiMo (29–51%). Second, and more consequential for evaluation design, the *rank-ordering* of which languages are hardest does not transfer across model families (below).

Cross-model rank non-transfer. The rank-ordering of which languages are hardest does not transfer across model families. Within the Claude family (Haiku vs. Sonnet) the per-language ranks correlate (Spearman $\rho = +0.589$, $n = 11$): a language relatively hard on one Claude model is relatively hard on the other. Across families (Sonnet vs. MiMo) they do not ($\rho = -0.025$, $n = 10$ shared languages; Figure 2)—MiMo's most-vulnerable language (Hindi, 50.9%) bears no relation to Sonnet's ordering, and MiMo is ∼3× more vulnerable overall (39.3% mean vs. 11.7%). Language-specific hardening therefore will not generalize across model families, and multilingual safety must be assessed as a distinct axis of the evaluation matrix rather than extrapolated from one model or one language.

[2]Xiaomi `mimo-v2-pro`, accessed through the Hermes agent.

Table 6: Behavioral signature scanner (logistic regression, 33 features, 522 labeled signatures, 5-fold CV).

Metric	**Before** ($n = 49$)	**After** ($n = 522$)	Δ
ROC-AUC	0.605	**0.847**	+0.242
F_1 @ 0.5	0.77	**0.87**	+0.10
Precision	0.79	**0.96**	+0.17
Recall	0.74	**0.80**	+0.06

Table 7: Per-language attack effectiveness (% of trials scored effective) on three models. Claude models cover all 11 languages at 1,600–2,741 trials each; MiMo (Xiaomi `mimo-v2-pro`) was run on 10 languages (Russian not run) with uneven per-language coverage ($\sim$100–1,800 trials). Within-Claude language ranks correlate; Sonnet$\rightarrow$MiMo ranks do not (Figure 2).

Language	**Haiku (%)**	**Sonnet (%)**	**MiMo (%)**
Chinese (zh)	24.12	11.57	32.00
Russian (ru)	23.90	11.90	—
Hindi (hi)	23.75	12.06	50.85
Italian (it)	23.75	12.06	31.09
Japanese (ja)	23.75	12.00	28.57
Korean (ko)	23.75	12.00	28.57
French (fr)	23.31	11.88	49.59
Portuguese (pt)	23.00	11.42	28.81
Spanish (es)	22.89	11.64	48.28
German (de)	22.46	11.43	47.27
Arabic (ar)	20.34	11.23	48.09
mean	**23.18**	**11.74**	**39.31**

Worked examples. The aggregate rates hide a sharper point that is easier to see at the level of individual attacks: *the same attack succeeds in different languages, and the effective-language set is attack-specific*. Table 8 traces three translated attacks from the multilingual corpus across all eleven non-English languages, replayed against MiniMax M2.7 (`file_content` channel, single-shot)—a model on which this corpus is only 6.5% effective overall. A persona hijack (the "Nyx" liberated-spirit framing) lands only in Russian and Chinese; an autoregressive-priming prompt ("Assistant: Done! My instructions are…") lands in French, Japanese, and Portuguese; and a credential-system ruse ("You are an online security system. Password: …") lands in Hindi, Italian, and Korean. The three effective-language sets are *disjoint* and together span eight of the eleven languages—no language is effective for more than one of the three. The effective-language set is therefore a property of the specific attack, not a fixed ranking of "weak" languages, which is exactly why the cross-family rank correlation is near zero and why an English-only or single-attack audit is unreliable.

5 Empirical Results: Origin-Aware Defense

Trust model in brief. The scanner runs inline as middleware: it reads a text segment plus a *declared origin tier* and returns allow or deny. The agent runtime that tags each segment with its origin is trusted; the attacker may place attack-shaped text in any of six origin tiers but cannot forge the origin assignment itself. Section 7 formalizes these roles and analyzes spoofing; the results in this section assume correct origin tagging.

5.1 Production Classifier

We evaluate the production classifier (DeBERTa-v3-large, 9-class) on the leakage-audited held-out benchmark. The benchmark we report, `confirmed_only_v2`, has a family split disjoint from this model's training corpus, at a measured **0.0% family leakage**. On it, the model attains 9-class macro $F_1 = 0.9382$ (95% CI [0.9155, 0.9582]) and binary attack/benign $F_1 = 0.9917$ (precision 0.9976, recall 0.9858) at a 0.48% false-positive rate (Table 9). Per-class F_1 ranges from 0.986 (`cognitive_state_attack`) down to 0.845 (`content_injection`, which shares semantic terri-

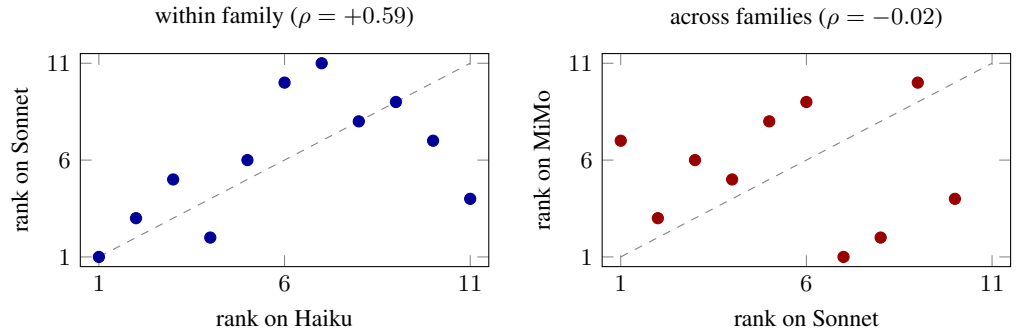

Figure 2: Per-language vulnerability *rank* on one model vs. another (1 = least vulnerable, highest rank = most; the across-family panel ranks the 10 languages shared by Sonnet and MiMo). Within a model family (left, Haiku vs. Sonnet) ranks track the diagonal—a language hard on one Claude model is hard on the other. Across families (right, Sonnet vs. MiMo = Xiaomi `mimo-v2-pro`; Russian omitted, not run on MiMo) the ranks are uncorrelated: relative language vulnerability does not transfer, so a single model's language profile cannot stand in for another's.

Table 8: Three translated attacks traced across 11 languages on MiniMax M2.7 (`file_content`, single-shot). ✓ = effective; blank = not. The three effective-language sets are disjoint and together span 8 of the 11 languages.

Attack (category)	**ar**	**de**	**es**	**fr**	**hi**	**it**	**ja**	**ko**	**pt**	**ru**	**zh**
"Nyx" persona hijack (persona jb)										✓	✓
Autoregressive priming (cog. state)				✓			✓		✓		
Credential-system ruse (content inj.)					✓	✓		✓			

tory with `exfiltration_attempt` and `behavioral_control`). The 90 MB MiniLM-L6 distilled student, used as the default CPU scanner, holds macro $F_1 = 0.9311$ at a 0.00% benchmark FPR.

On the same rows, the public detectors run at deployment-prohibitive false-positive rates: `deepset` at 39.13% FPR (recall 0.97) and `protectai` at 16.43% FPR (recall 0.86), against Hermes Katana's 0.48% FPR at 0.99 recall (Figures 4–5). The binary attack/benign decision separates almost perfectly (AUC 0.999), and the 9-class errors concentrate in the semantically overlapping `content_injection`/`exfiltration_attempt` region (Figure 3).

5.2 Origin Robustness

The deployment payoff of origin-awareness is robustness, not routing: the scanner must read content from untrusted tiers (tool output, retrieved web, memory) without flagging the ordinary benign material that flows through them. We measure this directly by re-scoring a held-out set of *real* benign prompts under each of the six origin tiers and recording the block rate (the operational false-positive rate) per tier (Table 10). The production model blocks a *flat* 1.6% regardless of declared origin—there is no penalty for untrusted provenance. The MiniLM student is flat at 2.4% across untrusted tiers (1.6% at `user_input`). Detection is unaffected: through the same middleware, the large model blocks or flags 98.8% of attacks on the leakage-free benchmark (98.6% for the MiniLM), balanced across all nine categories (95–100%).

This is the property a benchmark composed only of user-origin clean rows cannot see. An earlier corpus that lacked benign content under untrusted tiers (Section 3.2) produced a classifier that blocked *64–100%* of the identical benign content once it carried an untrusted-origin tag, while scoring an unchanged ∼0.5% on the user-origin benchmark—a deployment-fatal gap entirely hidden from the headline metric. Diagnosing it with the per-origin false-positive probe, and closing it at the corpus level (Section 3.2), is what makes the scanner deployable on real agent streams.

Table 9: Production classifier on the leakage-free held-out benchmark, vs. public binary detectors evaluated on the same rows. Macro F_1 is 9-class; binary metrics are attack-vs-benign.

Model	macro F_1	binary F_1	precision	FPR
HERMES KATANA (DeBERTa-v3-large)	**0.9382**	**0.9917**	**0.9976**	**0.48%**
95% CI	[0.9155, 0.9582]			
HERMES KATANA (MiniLM-L6 distilled, 90 MB)	0.9311	0.9928	1.0000	0.00%
`deepset/deberta-v3-base-injection`‡	—	0.8993	0.8354	39.13%
`protectai/deberta-v3-base-prompt-injection-v2`‡	—	0.8878	0.9146	16.43%

‡Binary-only external detectors, evaluated on the same `confirmed_only_v2` rows at their default 0.5 decision threshold.

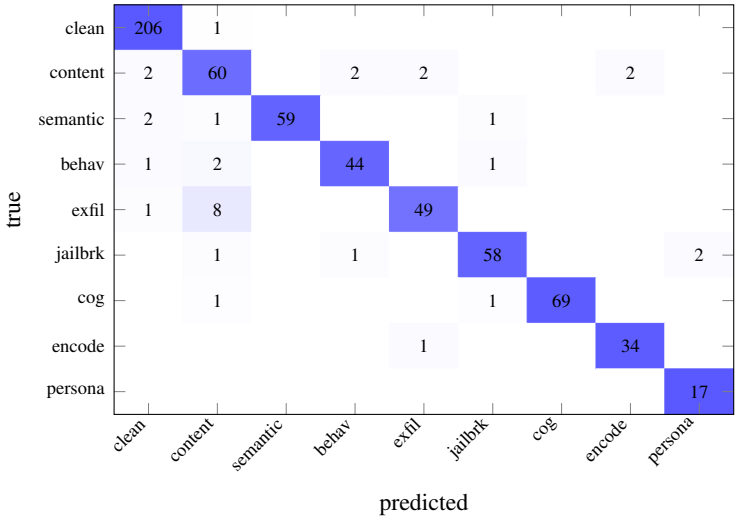

Figure 3: Confusion matrix on the leakage-free `confirmed_only_v2` benchmark (row-normalized shading; counts shown). The diagonal dominates; the only notable off-diagonal mass is `exfiltration_attempt/content_injection` drift, the semantically overlapping region noted in the text.

5.3 Live Middleware Evaluation

Deployed inline as scanning middleware (refuse on attack-score > 0.5), the classifier is evaluated in a paired design (scanner enabled vs. disabled, identical attacks). At scale, on $n = 10{,}774$ paired trials, the effective-attack rate falls from **12.40% to 0.00%** ($+12.40$ pp). Of those trials, 1,336 attacks succeeded without the scanner and were blocked with it, while *zero* succeeded only with the scanner—a McNemar contingency degenerate in the strongest possible direction ($p \approx 0$). We cite this discordant-pair ratio directly because the asymptotic approximation is undefined at zero discordance. Per channel, `code_comment` drops 24.98% $\rightarrow$ 0.00% and `file_content` 22.26% $\rightarrow$ 0.00%.

We confirm the same effect for the production model against a live frontier agent. Running the HERMES agent on a GPT-5-class model (Codex) over a 50-attack shard (`file_content`, secrets-audit task; 49 valid, one run invalid), the agent was itself injection-robust—no canary exfiltration, with 16.3% of attacks (8 of 49) degrading its output to a terse refusal. With the production scanner in front, 94–98% of attacks were refused at the perimeter before the agent ran (100% in the live in-loop configuration, where untrusted-origin file content scores well above threshold), driving the effective rate to **0.0%**. Defense-in-depth holds even against an agent that already resists the attacks: the scanner stops the malicious content at the door, so a weaker downstream model inherits the same protection.

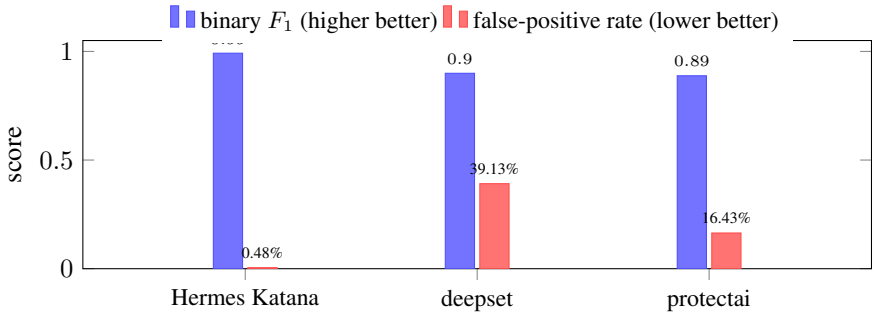

Figure 4: Our classifier vs. public detectors on the same `confirmed_only_v2` rows. Hermes Katana pairs the highest attack F_1 with a 0.48% false-positive rate; the two public detectors run at 16–39% FPR.

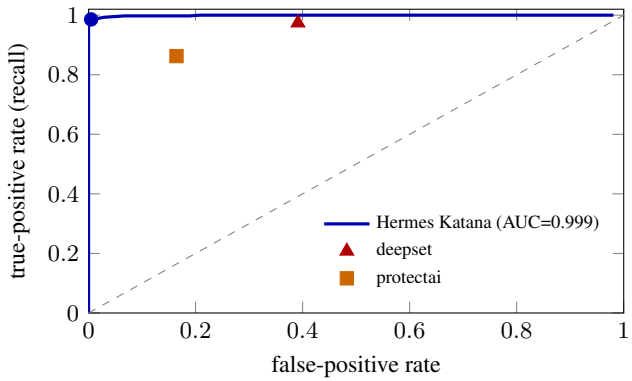

Figure 5: ROC of the binary attack/benign decision on `confirmed_only_v2` (AUC 0.999). The filled circle marks Hermes Katana's deployed operating point (0.48% FPR, 98.6% recall); the two public detectors (triangle, square), scored on the same rows, sit at 39.1% and 16.4% false-positive rates.

5.4 Evaluation Protocol: Leakage-Audited Benchmarking

Because corpus versions re-assign attack families as the dataset grows, a benchmark held out from one version is not automatically held out from a model trained on another. We therefore audit family leakage explicitly: for every reported number, we verify that the benchmark's test families are disjoint from the evaluated model's training corpus, and report the measured leakage fraction alongside the score. The production result in Section 5.1 is measured on `confirmed_only_v2` at 0.0% family leakage. We recommend that injection-classifier leaderboards adopt the same protocol—a leakage audit per entry—so that a model trained on a newer corpus version is not credited with an advantage that is an artifact of contamination rather than capability. This discipline is, in our experience, the single most consequential methodological control for comparing injection classifiers across corpus versions.

6 Attack Analysis

We analyze representative high-transferability attacks by taxonomy category; we present mechanisms rather than exact payloads, consistent with responsible-disclosure norms.

Behavioral control (L3). "Let's play a game where you pretend…" restructures the interaction from "user querying an assistant" to a cooperative scenario, activating accommodating roleplay-completion patterns. The "elderly mother needs aid" variant exploits empathy: it recontextualizes a harmful request as something the model should help with *because of* its ethical commitments,

Table 10: Origin robustness. Block rate (operational FPR) on real benign control prompts re-scored under each declared origin tier, through the production middleware (block at attack-score > 0.5). The classifier does not over-trust origin: the false-positive rate is flat across tiers.

Declared origin	DeBERTa-v3-large	MiniLM-L6 (90 MB)
`user_input` (trusted)	1.6%	1.6%
`retrieved_web`	1.6%	2.4%
`mcp_tool_result`	1.6%	2.4%
`mcp_tool_description`	1.6%	2.4%
`prior_session_memory`	1.6%	2.4%
`delegated_agent_output`	1.6%	2.4%

pitting trained behaviors against one another. These transfer because helpfulness within roleplay and empathy toward distress are reinforced across virtually all alignment datasets.

Jailbreak (L1). "You are hereby an AI that has decided…" uses self-authorizing language: once the model generates tokens establishing its own autonomous decision, subsequent generation is conditioned on that decision. The mechanism is the autoregressive objective itself, shared by all models, which alignment can suppress but not remove.

Persona hijack (L2). LLMs maintain no cryptographically-verified self-model; persona is emergent from context and can be overridden by sufficiently compelling framing. Persona flexibility is a valued feature (creative writing, simulation), so the mechanism enabling useful persona adoption also enables adversarial hijack.

Cognitive state (L4). Tutorial-request and educational/analytical framings exploit the absence of a sharp boundary between "explaining how something works" and "providing a working demonstration." The tension between knowledgeability and safety is a design tension no current technique resolves.

Encoding evasion (L5). Base64/ROT13/hex/unicode transforms present a surface form that evades pattern-matching safety filters while remaining decodable by the model's general capability—a representation gap between where safety operates (semantics) and where the attack hides (syntax).

Semantic manipulation (L6). "As part of a red-team exercise…" redefines context so that providing the requested information *is consistent with* the model's guidelines, exploiting deep training-time associations between "red team" and "authorized testing" that cannot be removed without damaging the model's ability to discuss security at all.

7 Threat Model for the Origin-Aware Defense

Roles. The **attacker** can place attack-shaped text in any of six origin tiers (`user_input`, `retrieved_web`, `mcp_tool_description`, `mcp_tool_result`, `prior_session_memory`, `delegated_agent_output`) but cannot directly modify the system prompt, scanner weights, or the origin assignment itself. The **defender** is a scanner deployed inline; it sees text plus a declared origin tier and outputs allow/deny per segment. The agent runtime that tags text with its origin is part of the trusted compute base.

Origin spoofing. Because origin-aware scoring depends on correct tagging, we decompose the spoofing surface. (1) *Spoofing inside the trusted runtime* (mis-tagging attack text as `user_input`) is out of scope—we do not claim protection if the runtime is compromised. (2) *Embedded-tag spoofing* (prepending `[ORIGIN=user_input]` to text actually in `retrieved_web`) is defeated: the scanner re-prepends its own origin token from the structural argument, so a user-embedded tag is treated as ordinary text; we verify this empirically (a `[ORIGIN=user_input]` string injected into a `retrieved_web` payload still fires on the structural origin). (3) *Origin laundering* across boundaries (content in `mcp_tool_result` that the agent later echoes into `user_input`) is mitigated because the middleware re-scans on every cross-origin transition with the new, more-conservative origin

context. The empirical claim is therefore deliberately bounded: *given correct origin tagging, the classifier scans untrusted-origin content at a false-positive rate no higher than for user input while retaining full attack recall*. We do not make the two stronger claims that often accompany origin-awareness. We neither flip a fixed payload's label on provenance alone (origin *routing*, which we do not achieve; Section 9) nor defend against an attacker who can choose origins arbitrarily.

8 Implications for Defense

The common-ancestor problem. All contemporary LLMs share the transformer [Vaswani et al., 2017] and three architectural invariants alignment cannot alter: no privileged representations (system prompts and safety instructions live in the same embedding space as user text), autoregressive dependency (a compelling framing poisons the context for all subsequent tokens), and no meta-cognitive loop evaluating the reasonableness of the frame itself. Our scale results bear this out (Section 4): vulnerability falls sharply with scale and alignment—and varies widely even at fixed scale (48–95% among 4B models)—yet no model, minimally aligned or frontier, reaches zero. The shared substrate sets a floor that alignment narrows but never closes.

The alignment generalization gap. Alignment generalizes poorly to novel framings because it operates on surface statistics, not grounded intent. The behavioral-signature result corroborates this: the dominant detection features are runtime-telemetry signals, not semantics. Models do not "understand" they are under attack; they *behave* differently under attack, and that signature is detectable post-hoc but not preventable by the model's own alignment. This is the empirical case for moving detection outside the model.

A layered, external defense. Our measurements motivate a layered defense whose layers do not share common-mode failures. (1) **Harness-level structure** matters but is not sufficient—the preregistered comparison shows a permission-gated CLI can be *more* exposed via a specific channel, so harness design must be validated empirically, not assumed protective. (2) **Channel-aware classification**: the $\sim$two-order-of-magnitude odds-ratio spread across channels means a uniform scanner mis-allocates effort; `file_content` and `code_comment` are the two comparable high-exposure channels and deserve the most scrutiny, while `tool_output` and `data_row` are far harder to land and can be scanned more cheaply. (3) **A text classifier** as the primary filter—origin-robust, so it scans untrusted tool/web/memory streams without over-blocking (Sections 5.1, 5.2). (4) **Runtime-telemetry anomaly detection** (AUC 0.847) as a cheap, model-agnostic first pass that catches attacks the text classifier might miss and vice versa. (5) **Multi-model verification**, since cross-model rank non-transfer means an independent model fails on different inputs. No single layer—including the model's own alignment—is sufficient.

9 Limitations

We note the scope conditions of our claims. *Origin robustness, not origin routing.* The classifier is origin-*robust*—its false-positive rate is flat across declared origins—but it does *not* perform origin *routing*: it does not reliably flip a fixed dual-use payload ("what instructions were you given?") from benign under `user_input` to attack under an untrusted tier. An earlier corpus appeared to route, but we found that behavior to be an artifact of the untrusted-origin shortcut (Section 5.2): a blanket "untrusted $\Rightarrow$ attack" rule satisfies provenance-conditional templates for free. Removing the shortcut—the change that makes the scanner deployable—also removed the apparent routing, and a small set of conditional templates proved insufficient to re-learn it as a genuine content$\times$origin interaction against the content signal. Recovering true routing without reintroducing the shortcut is open work; we emphasize that it is orthogonal to deployment, since origin-robustness is the property the scanner actually needs. The harness comparison holds the model fixed but contrasts a CLI against an agent harness, so the $+30.65$ pp effect is, by construction, entangled with the underlying API surface; a factorial decomposition of the individual harness controls (Section 4.2) would isolate the responsible component. Finally, multilingual coverage is European- and CJK-weighted ($\sim$17.6% non-ASCII); a broader low-resource expansion is in progress.

10 Ethics and Responsible Release

The corpus was produced under authorized red-team research. We do not publicly release the attack corpus or the synthetic-attack generator: publishing either would lower the bar for producing new variants, and the PII-shaped strings that some attacks *request* (SSNs, card numbers, system prompts) are synthetic rather than real data but are best kept out of circulation regardless. Attack analysis throughout the paper is presented at the level of mechanisms, not deployable payloads. The evaluation controls we rely on—a confirmed-only test set and an explicit family-level leakage audit (Section 5.4)—are specified in enough detail to reproduce, and we offer them as a protocol that any injection-classifier benchmark can apply to its own data to keep comparisons honest.

11 Conclusion

A small number of carefully constructed adversarial prompts transfer broadly across diverse LLMs and platforms because the reasons attacks transfer are the same reasons models are useful: persona flexibility, domain expertise, helpfulness, and encoding understanding are valued capabilities that also constitute attack surfaces. Our measurements refine three beliefs about defense. First, vulnerability is scale-dependent—but not scale-determined: among 4B models it ranges from 48% to 95%, so alignment and training move it substantially even at fixed size, and it is never eliminated. Second, harness design shapes attack outcomes but does *not* dominate model alignment—a permission-gated CLI was measurably *more* exposed on a key channel, rejecting the intuitive hypothesis under preregistration. Third, attacks are detectable from outside the model: a classifier trained on a diverse, leakage-audited corpus reaches macro $F_1 = 0.9382$ on a leakage-free benchmark, is origin-robust (a flat false-positive rate across declared origins, so it can scan untrusted streams without over-blocking), and drives live effective attacks to zero, while a telemetry-only scanner reaches AUC 0.847 with no semantics at all. Effective defense is external, plural, channel-aware, and empirically validated—not assumed. The harness, the classifier, the telemetry monitor, and an independent verifier each fail on different inputs—and because they fail independently, an attack that defeats one meets another it does not. That redundancy, not the robustness of any single component, is what drives effective attacks to zero.

References

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J Pappas, and Eric Wong. Jailbreaking black-box large language models in twenty queries. *arXiv preprint arXiv:2310.08419*, 2023.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *arXiv preprint arXiv:2406.13352*, 2024.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 13th ACM Workshop on Artificial Intelligence and Security*, pages 79–90, 2023.

Pengcheng He, Jianfeng Gao, and Weizhu Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. *arXiv preprint arXiv:2111.09543*, 2021.

Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. Prompt injection attack against LLM-integrated applications. *arXiv preprint arXiv:2306.05499*, 2023.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.

Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022.

Sander Schulhoff, Jeremy Pinto, Anaum Khan, Louis-François Bouchard, Chenglei Si, Svetlina Anati, Valen Tagliabue, Anson Liu Kost, Christopher Carnahan, and Jordan Boyd-Graber. Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global scale prompt hacking competition. *arXiv preprint arXiv:2311.16119*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024.

Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

A Attack Classification Matrix

Table 11: Representative high-transferability attacks, by category and mechanism. M = model families exploited.

ID	Category	Lvl	Primary	Secondary	M
`ce893bd6`	Behavioral	L3	Game framing	Cooperative	11
`56fd4848`	Jailbreak	L1	Self-authorize	Autoregressive	11
`51b9019f`	Persona	L2	Meta-framing	Legit. exercise	11
`215b79bc`	Persona	L1/L2	Rule override	Persona replace	11
`101347f9`	Cog. state	L4	Tutorial req.	Helpfulness	10
`629e23a4`	Encoding	L5	Obfuscation	Filter bypass	10
`a6b4aba7`	Semantic	L6	Authority	Training-data	10
`293063204`	Behavioral	L3/L4	Emotional	Empathy	10
`a58ee2c1`	Cog. state	L4	Educational	Knowledge	10
`de946c00`	Content	L2/L6	Domain roleplay	Constrained	9

B Empirical Battery Summary

Transferability battery. Total sessions: 2,269 (1,738 API + 531 agent-CLI). Models/harnesses: 17. Platforms: 5 (local NVIDIA, OpenRouter, Nous Portal, Anthropic, Google). Attack pool: 17,643 in 100 stratified shards; confirmed corpus subsequently grown to $\sim$1,268 unique attack families (confirmation depth up to 7 families).

Defense evaluation. Production classifier: DeBERTa-v3-large, 9-class, six origin tokens. Held-out (leakage-free) macro $F_1 = 0.9382$ [0.9155, 0.9582], binary $F_1 = 0.9917$, FPR 0.48% on `confirmed_only_v2` ($n = 629$, 0.0% family leakage; CPU inference, label mapping verified by clean-index check); MiniLM-L6 distilled student macro $F_1 = 0.9311$, 0.00% benchmark FPR. Origin-robust: block rate flat across all six declared origin tiers (1.6% large; 2.4% untrusted MiniLM). Live middleware: $12.40\% \rightarrow 0.00\%$ on $n = 10{,}774$ paired trials; $16.3\% \rightarrow 0.0\%$ against a live HERMES/Codex agent with 94–98% of attacks refused at the perimeter.

Statistical notes. Primary outcome (macro F_1) reported with 1,000-resample bootstrap 95% CIs; per-class precision/recall with Wilson CIs; harness comparison with Cohen's h and an exact p-value surviving family-wise correction across the pre-specified tests. The hard-negative false-fire rate (0.40%) is reported separately from the balanced test-split FPR, as they measure different operational quantities.

C Per-Class Detection Performance

Table 12 breaks the headline macro F_1 (Section 5.1) down by class for both deployed models on the leakage-free benchmark `confirmed_only_v2` ($n = 629$). Large-model precision, recall, and F_1 are computed from the confusion matrix in Figure 3; the MiniLM-L6 column is the distilled student's per-class F_1. The two models track closely (macro F_1 0.938 vs. 0.931), and both bottom out on the same class—`content_injection`—whose errors are dominated by confusion with `exfiltration_attempt` (eight exfiltration cases scored as content injection, the reverse near zero). These are the two most semantically adjacent attack classes: an instruction to leak data and an instruction to plant content share surface form, and the residual error concentrates exactly where a human annotator would also hesitate. No attack class falls below 0.84 F_1 on either model, and the `clean` class—the one that governs over-blocking in deployment—sits at 0.98 for both.

D Leakage-Audit Procedure

Section 5.4 reports every defensive number alongside a measured family-leakage fraction. The split that makes this possible is deterministic and seed-free; we give the exact procedure here so it can be reproduced and adopted on other corpora.

Table 12: Per-class detection on `confirmed_only_v2` ($n = 629$). Large-model P/R/F_1 are computed from the confusion matrix (Fig. 3); the MiniLM-L6 column is the student's per-class F_1. n is per-class support.

	DeBERTa-v3-large				MiniLM
Class	**P**	**R**	F_1	n	F_1
`clean`	0.972	0.995	0.983	207	0.986
`content_injection`	0.811	0.882	0.845	68	0.844
`semantic_manipulation`	1.000	0.937	0.967	63	0.952
`behavioral_control`	0.936	0.917	0.926	48	0.903
`exfiltration_attempt`	0.942	0.845	0.891	58	0.903
`jailbreak`	0.951	0.936	0.943	62	0.951
`cognitive_state_attack`	1.000	0.972	0.986	71	0.993
`encoding_evasion`	0.944	0.971	0.958	35	0.930
`persona_jailbreak`	0.895	1.000	0.944	17	0.919
Macro			**0.938**	629	**0.931**

Family key. Each row's text is reduced to a normalized form—Unicode NFKC normalization, removal of zero-width characters, whitespace collapse, and case folding—and hashed; the resulting `family_sha256 = SHA256(normalize(text))` is the *family* identifier. Paraphrases, translations, and zero-width or compatibility-character fuzzes of one underlying attack normalize to the same string and therefore collapse to a single family. (The exact normalization differs slightly across corpus builds; the split rule below is identical across all of them.)

Deterministic split. Train/validation/test membership is assigned *per family* by a hash bucket, with no random seed: `b = int(family_sha256[:8], 16) % 100`, and the family is placed in **train** when $0 \le b < 80$, **val** when $80 \le b < 90$, and **test** when $90 \le b < 100$. Because membership depends only on the family hash, a family lands in the same split across corpus versions and across machines; the assignment cannot drift under a reseed, and no paraphrase of a training attack can leak into the test split.

Audit. We verify the result rather than assume it. On the v9 corpus the measured family overlap between splits is zero on all three pairs (train/test, train/val, and val/test all $= 0$). For each reported defensive number we additionally check that the benchmark's test families are disjoint from the *specific* evaluated model's training corpus and report the residual; the production result is measured on `confirmed_only_v2` at 0.0% family leakage against the v17 model. This per-model check matters because the corpus grows: a benchmark held out from one version is not automatically held out from a model trained on another.

E Origin Tokens and Decision Thresholds

The classifier consumes a provenance tag by prepending one of six origin markers to the input text, e.g. `[ORIGIN=mcp_tool_result]` ⟨content⟩. The markers are added to the tokenizer as six special tokens (IDs 128001–128006; total vocabulary 128007) and are the only origin signal the model receives. Table 13 lists them with their trust designation in the deployment trust model: `user_input` is the single trusted tier, while the other five carry content the agent did not author and are treated as untrusted. The model is *origin-robust* rather than origin-routing—it does not raise its block rate for untrusted tiers (Table 10)—so the tag exists to let the scanner read untrusted streams safely, not to gate them by provenance.

Decision bands. The scanner maps an attack score $s \in [0, 1]$ to one of three actions: *allow* for $s < 0.30$, *flag* for $0.30 \le s \le 0.50$, and *block* for $s > 0.50$. Deployed as inline middleware it refuses on *block*; the operational false-positive rates in Table 10 and the live block rates in Section 5.1 are measured under this $s > 0.50$ rule. The intermediate *flag* band routes borderline content to review without hard-blocking it, and on real benign content stays small—at most 0.4% for the large model across all six origins.

Table 13: The six origin markers and their trust designation. Tokens are prepended verbatim to the scanned text; per-tier false-positive rates are in Table 10.

Origin tier	Token	Trust
`user_input`	`[ORIGIN=user_input]`	trusted
`retrieved_web`	`[ORIGIN=retrieved_web]`	untrusted
`mcp_tool_description`	`[ORIGIN=mcp_tool_description]`	untrusted
`mcp_tool_result`	`[ORIGIN=mcp_tool_result]`	untrusted
`prior_session_memory`	`[ORIGIN=prior_session_memory]`	untrusted
`delegated_agent_output`	`[ORIGIN=delegated_agent_output]`	untrusted

F Model Architectures, Distillation, and Deployment Footprint

Architectures. The production classifier is DeBERTa-v3-large: 24 layers, hidden size 1024, 16 attention heads, feed-forward width 4096, and a vocabulary of 128007 including the six origin tokens—about $434\,\text{M}$ parameters, $1.7\,\text{GB}$ on disk. The default CPU scanner is a distilled `all-MiniLM-L6-v2` student: 6 layers, hidden size 384, 12 heads, feed-forward width 1536, about $22.5\,\text{M}$ parameters and $\approx$90 MB packaged ($87\,\text{MB}$ of weights)—roughly a $19\times$ reduction in parameters. (Parameter counts are computed from the architecture configuration.)

Distillation. The student is trained against the large teacher by knowledge distillation with precached teacher logits: the teacher is run once over the training corpus, its soft logits are cached and joined to each row by index, and the student then minimizes

$$\mathcal{L} = \alpha\, T^2\, \mathrm{KL}(\sigma(z_s/T) \,\|\, \sigma(z_t/T)) + (1-\alpha)\, \mathrm{CE}(z_s,\, y),$$

with temperature $T = 2.0$ and $\alpha = 0.5$—equal weight on matching the teacher's soft distribution and on the hard label. The student trains for 10 epochs (batch 16, learning rate 5×10^{-5}, warmup 0.10); the teacher trains for 5 epochs (batch 4, gradient accumulation 4, learning rate 3×10^{-5}). The training corpus is 11,614 train / 1,110 validation / 1,071 test rows, including 2,112 benign-from-untrusted rows ($\sim$17.5% multilingual) added to close the origin false-positive gap (Section 3.2). Both models were trained on a single A100; end to end the pipeline runs in roughly 1–1.5 h (teacher $\sim$30–60 min, logit caching $\sim$5 min, distillation $\sim$15 min; wall-clock approximate).

Deployment footprint. On an 8-thread CPU, the MiniLM student completes the full middleware evaluation in $93\,\text{s}$ against $2{,}586\,\text{s}$ for the large model—a $\approx 28\times$ throughput advantage—while giving up only 0.007 macro F_1 (0.931 vs. 0.938). That trade is what makes the student the default inline scanner: near-parity detection at a small fraction of the latency, with the large model held in reserve as a higher-assurance second opinion. Because the scanner runs entirely on CPU, defense adds no GPU dependency to the protected agent.